

RAG-Based Question Answering System

Freshers / Entry-Level AI Engineer Assessment

Duration	Level	Submission	Focus
Flexible	Fresher	GitHub + README	Practical RAG

1. Assessment Objective

Build a simple, working Retrieval-Augmented Generation (RAG) application that allows a user to provide documents, ask questions about them, retrieve relevant information, and generate answers grounded in the provided documents.

The expected flow is:

Documents → Text Extraction → Chunking → Embeddings → Vector Store → Retrieval → LLM → Answer + Sources

2. Problem Statement

Create a document-based Question Answering system. The application should answer questions using information found in the supplied documents and should avoid making up information when the answer is not available.

Example:

Document: Employee Policy.pdf

Question: How many days of annual leave are employees entitled to?

Expected: A grounded answer with the relevant document/page as the source.

3. Functional Requirements

A. Document Ingestion

- Support PDF and at least one additional text-based format such as TXT or Markdown.
- Extract text from uploaded/provided documents.
- Split the extracted text into meaningful chunks.
- Generate embeddings for the chunks.
- Store the embeddings in a vector database.

B. Retrieval

- Convert the user's question into an embedding.
- Retrieve the most relevant chunks from the vector store.
- Use Top-K retrieval or an equivalent retrieval strategy.
- Explain the chosen K value and retrieval approach in the README.

C. Answer Generation

- Pass the retrieved context to an LLM.

- Generate a concise answer grounded in the retrieved context.
- Do not invent information that is not supported by the documents.
- If the answer cannot be found, clearly state that the information is not available in the provided documents.

D. Source / Citation

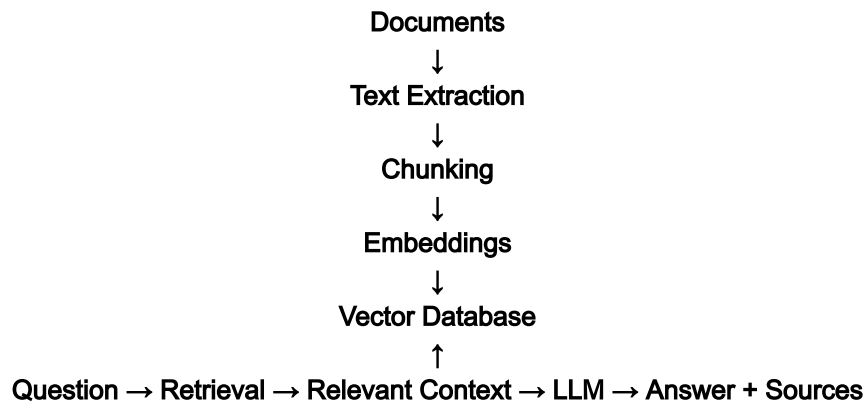
The response should identify the source wherever possible, for example: Employee_Policy.pdf — Page 8.

E. User Interface

Provide a simple interface using Streamlit, Gradio, Flask/FastAPI with a basic frontend, or an equivalent technology.

- Upload/select documents
- Enter a question
- View the generated answer
- View the source(s)

4. Expected RAG Architecture



5. Test Scenarios

Scenario	Condition	Expected Result
Direct question	Answer exists clearly in one document.	Return the correct grounded answer.
Multi-document question	Information is available across multiple documents.	Retrieve and combine relevant information appropriately.
Information unavailable	The answer does not exist in the documents.	State that the information was not found; do not hallucinate.
Similar information	Documents contain similar topics or competing details.	Retrieve the correct supporting context.

6. Technology Choices

You may choose the technologies and libraries you are comfortable with. Examples include:

- Python
- LangChain or LlamaIndex

- FAISS, Chroma, Qdrant, or another vector database
- OpenAI or Hugging Face embeddings
- OpenAI or another suitable LLM
- Streamlit, Gradio, FastAPI, Flask, or equivalent

You are not required to use the examples above. The important requirement is that you can explain your technical choices.

7. Submission Requirements

1. Complete source code in a GitHub repository.
2. README containing project overview, architecture, setup, environment variables, run instructions, technologies, chunking strategy, embedding model, vector store, retrieval strategy, LLM, and limitations.
3. 3–5 sample documents for testing.
4. Screenshots or a short demo showing document upload, question, answer, and source.
5. At least 10 test questions: 5 answerable, 2 requiring multiple documents, and 3 unanswerable.

8. Evaluation Criteria — 100 Marks

Area	Marks
Document ingestion	10
Chunking strategy	10
Embeddings	10
Vector database	10
Retrieval	15
LLM integration	10
Grounded answers / hallucination handling	10
Source citations	5
UI / usability	5
Code quality	5
README / documentation	5
Testing	5
TOTAL	100

9. Achievement Levels

Bronze — Basic RAG

Working document ingestion, chunking, embeddings, vector store, retrieval, and LLM response.

Silver — Reliable RAG

Adds source citations, multiple documents, unanswerable-question handling, configurable Top-K, and basic error handling.

Gold — Advanced RAG

Adds improvements such as hybrid search, reranking, metadata filtering, query rewriting, retrieval evaluation, or similarity thresholds.

Platinum — Production Thinking

Demonstrates evaluation, metrics, logging/observability, latency and cost considerations, testing, Dockerization, or API-based architecture.

10. Viva / Technical Discussion

Be prepared to explain your implementation

11. Bonus Challenge

If you complete the core assessment early, add conversational RAG. The system should understand follow-up questions using previous conversation context while still grounding answers in the documents.

12. What We Expect

This assessment is not intended to test how many frameworks you know. We want to understand whether you can build and explain a complete RAG pipeline, make sensible engineering decisions, handle cases where information is unavailable, and produce a clean, reproducible implementation.

Good luck!